

StockKarpador

TeamBuildPolicy

The HeuristicTeamBuildPolicy implements a **heuristic-based approach** for constructing teams through a blend of random sampling and statistical evaluation. The system operates in three main phases:

1. candidate generation
2. team evaluation
3. command construction.

Candidate Generation

The candidate generation phase employs a sampling approach that creates multiple potential team compositions by randomly selecting Pokémon from the available roster. Rather than directly evaluating the entire roster and simply choosing the top-performing individual Pokémon, the policy generates a configurable number of candidates (defaulting to 20) to explore diverse team combinations. This randomized selection strategy serves as a deliberate mechanism to prevent overfitting to simplistic evaluation metrics by introducing stochastic exploration into the team building process. By sampling different combinations rather than greedily selecting the highest-rated individuals, the system implicitly creates opportunities for discovering beneficial team synergies such as type coverage and role complementarity, without requiring the computational overhead of explicitly modeling these complex interactions.

Team Evaluation

The core of the HeuristicTeamBuildPolicy is the `evaluate_team` function. This function assigns a numerical score to a potential team, allowing for a quantitative comparison between different team compositions. The evaluation metric is a straightforward yet effective heuristic:

- **Base-Statistic Score:** For each Pokémon on the candidate team, the policy calculates the sum of its base stats (HP, Attack, Defense, Special Attack, Special Defense, and Speed). This provides a measure of the Pokémon's raw statistical potential.
- **Move Score:** The policy also considers the offensive potential of each Pokémon's moves. For each move, a simple damage metric is calculated by multiplying its `base_power` by its `accuracy`. The sum of the scores of the four most powerful moves is then added to the Pokémon's total score.

The total score for a team is the sum of the individual scores of its six members. This approach prioritizes Pokémon with high all-around stats and powerful, reliable attacks.

Command Construction

The command construction phase translates the highest-scoring team into executable build instructions. For each selected Pokémon, the system randomly distributes EVs using a multinomial distribution across all six stats, sets IVs to maximum values (31 across all stats), randomly selects a nature, and initializes moves.

In conclusion, the HeuristicTeamBuild Policy represents a foundational approach which takes very little computational costs. Future iterations could extend the evaluation metric to incorporate more sophisticated factors, such as type synergy analysis that considers elemental matchups and coverage, as well as team role balancing that ensures proper distribution of strategic archetypes e.g tanks, healers, damage dealers, and control specialists. Another significant opportunity for improvement lies in implementing battle simulation testing. Rather than relying solely on theoretical evaluations, future versions could evaluate teams through actual battle scenarios and select based on win rates. Integrating the battle policy directly into these simulations would create a synergistic optimization process, ensuring that team composition and battle strategy are aligned to maximize competitive performance together.

Selection Policy

The Selection policy uses a simple net-effectiveness heuristic to choose the best n Pokémon against the opponent.

Calculating Effectiveness

For each of our Pokémon, the policy calculates an effectiveness_score based on its potential matchups against all of the opponent's Pokémon. This score is determined by two components:

- **Offensive Score:** This is the sum of the maximum possible damage our Pokémon can inflict on each of the opponent's Pokémon assuming no status changes or other influencing factors as a simplification. The calc_max_dmg function is used to simulate a one-on-one scenario and find the damage of the most powerful move our Pokémon has against each opponent.
- **Defensive Score:** Conversely, this is the sum of the maximum possible damage each of the opponent's Pokémon can inflict on our Pokémon given the same conditions.

The final effectiveness_score is calculated as:

$\text{effectiveness_score} = \text{offensive_score} - \text{defensive_score}$

This should provide a balanced view of each Pokémon's viability in the upcoming battle by rewarding Pokémon that have high damaging moves against the specific opposing team while also being able to withstand the opponent's attacks.

Final Selection

After calculating the effectiveness_score for all of our Pokémon, they are ranked from highest to lowest score. The top n Pokémon from this list are then selected to form the team for the upcoming battle.

Limitations

This approach ensures that our team is tailored to exploit the specific weaknesses of the opponent's team composition, but we are not considering many variables that might have a significant influence on the viability. E.g. We are not considering any non-damaging moves and how they might influence the game based on long-term-decision making. This is a simplified approach that is mostly meant to filter out Pokémon that are for some reason unviable against a certain team. The Team Build Policy is supposed to create a team that is already viable on a lower lever, while the selection policy checks for the Pokémon that lose a significant amount of power when they are up against an opponent that is e.g. considered a "counter" i.e. a Pokémon with specific strengths against certain other Pokémon.

Monte Carlo Battle Policy

The original form of this policy used to be more complex, but we noticed that the algorithm had to be simplified. Operating under a strict 100ms time limit per turn, a full Monte Carlo Tree Search (MCTS) would need too much computational power to be viable. Instead, we use a flat Monte Carlo approach, which involves sampling a set of possible actions, simulating their outcomes to a limited depth, and selecting the action that leads to the best state.

Decision Making

The decision-making process for each turn involves the following key components:

1. **Action Generation**
2. **Simulation**
3. **Parallelization**
4. **Evaluation**

The biggest influence on runtime comes from the number of simulations that are run. With the right parameters, this process allows us to make decisions within the given time constraint.

Action Generation

We generate a list of possible actions (`full_actions`) using the `get_actions` function. To limit the search space, it only generates a list of all possible non-attacking actions (e.g., status moves, switching out) for our active Pokémon, while attacking moves are handled differently. We filter out the “pure attacking moves” e.g. those that deal a certain amount of damage but do not apply any kind of status- or environmental- effect, alter the stats etc. for those pure attacking moves, we instead use the greedy policy as a baseline. We only do the simulation once for the greedy decision applied a given number of times in a row instead of comparing each pure attacking move because we think that it is irrelevant to look at them due to the fact that some pure attacking moves should be purely worse than the greedy move.

Simulation

This step was originally meant to be more complex but later cut down in the number of simulations to save on time. The simulation function is used to take a given action, copy the state, and then run the simulation with the given action in the first step, then use random actions on following steps. we changed it to use a greedy action on subsequent steps instead of a random strategy to eliminate the randomness. To simplify and limit the size of the search space even more, our policy operates under the key assumption that the opponent will play according to a `GreedyBattlePolicy`. While this assumption is not always accurate, it provides a reasonable and computationally inexpensive baseline for predicting the opponent's actions. Before making a decision, the `deduce_state_safe` function is used to update our internal representation of the opponent's team [this is simply the `deduce_state` function from `vgc2`, but with a bugfix that prevents sporadic bugs.] After the desired depth is reached, we return the state value calculated by the `evaluate_state_with_context` function

Parallel Evaluation

For each action the simulation is called a set number of times and the average value (coming from the evaluation function) will be maximized. In the final implementation we decided to go with only one simulation for all actions. This is why we changed the random policy to greedy policy.

To manage the time constraint, these simulations are run in parallel using a `ThreadPoolExecutor`. All actions that are not purely attacking are used as an initial action and run by the simulation function. Additionally, one greedy action is always simulated by default.

State Evaluation

The state evaluation is arguably one of the most important components of a battle policy because it quantitatively evaluates a given game state, which is needed to compare different choices without simulating too far into the future. The state evaluation has to indicate clearly if one state is superior to another only by looking at the values given in a state. The `eval_state_with_context` function assigns a score to a state based on a weighted combination of the following factors:

- **Team HP:** The normalized remaining HP of our team.
- **Opponent's Team HP:** The normalized remaining HP of the opponent's team.
- **Alive Pokémon:** The number of conscious Pokémon remaining on our team.
- **Opponent's Alive Pokémon:** The number of conscious Pokémon remaining on the opponent's team.

The evaluation function is defined as:

$$\text{score} = (3 \times \text{my_hp}) - (5 \times \text{opp_hp}) + (2 \times \text{my_alive}) - (2.5 \times \text{opp_alive})$$

This formula is designed to prioritize our own team's survivability while also rewarding actions that reduce the opponent's resources. The higher weighting on the opponent's HP incentivizes offensive play. This is simply a weighting that we found to have great success and is not based on any kind of comprehensive evaluation.

Limitations

The evaluation is simplified to only use the four given components. The reason for this is that a higher number of factors would increase the complexity drastically. Without a sound modeling technique, it is nearly impossible to find factors that are important and weigh them appropriately. This is why we chose to use a known method that is proven to work. We then fine tuned the weighting by hand. The weighting is simply one that we found to have great success and is not based on any kind of comprehensive evaluation.

Choice of parameters

For each non-attacking action in the generated list, the simulation function is called one time. This function executes the chosen action and the opponent's predicted greedy action, then

simulates one further turn (our greedy move and the opponent's subsequent greedy move). Although limited in depth due to time constraints, the two-timestep lookahead provides a more accurate assessment of the board state than the simple greedy decision because we are also considering moves that do not deal damage but influence the game in a positive way regardless.